

THE COVERAGE JOB, RE-EXAMINED — what stands, what was wrong, and the amended solution

Version 3.40.0 · 2026-08-05 · Written by the routing session at Will's request (*"please re examine the study and design opus came up with"*, then *"okay go for it"*). This is the review AND the record of the amendments applied to `docs/ENGINE-COVERAGE-PLAN.md` in the same pass. Every number in here is read from an artifact on disk tonight, not from the plan's prose.

1. The verdict

The design is right and is kept. It is amended, not rebuilt. The diagnosis is correct, the instruments are the right ones, and the plan is honest in a way plans rarely are — it names its own traps and demands every check be shown failing before it is trusted. But it contained two internal contradictions and three concrete implementation traps, and the data that arrived tonight deflates one layer and invalidates the endgame as written. All are fixed in the amended plan.

The literature is used correctly, which was checked rather than assumed: the tag system is the data-direction of Wadler's Expression Problem (1998) and is only safe with an exhaustiveness check; the producer–consumer coupling is Page-Jones's connascence of meaning; the mutation tier is DeMillo, Lipton & Sayward (1978). Two attributions the original omitted, added for the record: Layer 1's byte-identical gate is a **golden-master (characterization) test**, and Layer 4 is **differential testing** in McKeeman's (1998) sense. Nothing was mis-cited.

2. What tonight's data settled before the review even started

Question the plan left open	Answer, from the artifact
Do the two rulebooks disagree?	2 clashes in 151 comparable facts (<code>data/rulebook-collision.json</code> , ratcheted ≤ 2)
The 68 matrix disagreements	13 remain — 55 resolved by wires 82–89, reproduced on a fresh <code>--full</code> run
The 7 missing mechanics	5 remain , each with its reason in the census (181/186 live)
The pre-turn class	Landed (wire 82). Shell Trap is banned in this format (<code>isNonstandard: 'Past'</code>) — its "untagged" state is the format door working, not a gap; the derivation already matches it in the full dex if a future regulation legalizes it
The 58-dim step rule	Fixed AND proven on a planted optimum — see §6, because the proof changes the endgame

The Iron Head clash is the whole two-rulebook story in one row: `tags.json` carried the Champions format's 20% flinch, `move-effects.json` carried the generic 30%, **and the engine was reading the wrong one.** Wire 89 now reads the format's own number and counts future drift (`MEDFAILS.rulebookChanceDrift`). The other clash (Toxic Thread, 6 uses) is the same shape smaller.

3. The two contradictions, and how the amendment resolves them

The two documents disagreed on the order of registry and mutation. `ENGINE-COVERAGE-PLAN.md` ran registry (Layer 2) before mutation (Layer 3); `TAG-COVERAGE.md` §4 said the opposite. One had to be wrong, and it was the plan, because:

The plan's stub defense did not work. It claimed a stub handler "shows up in `asked()` / `hits()` as a live tag with zero reads, and `test-tag-consumed.js` ratchets it." Trace that through the four-state table: a registered handler that never fires is *named in source with ASKED = 0* — that is **UNREACHED**, and TAG-COVERAGE explicitly refuses to ratchet UNREACHED, for the stated reason that it measures the sweep rather than the engine. A stub that fires but ignores its payload reads as LIVE. Stubs land in exactly the buckets the ratchet does not guard. **The only instrument that catches a stub is mutation. Therefore mutation ships before the registry.** The amended plan swaps the layers and says why.

4. Three traps in the mutation tier as originally specified

1. **The harness as written failed its own gate.** The validation case was `spreadAll`'s ignored `hitsAlly` param — but the specified operator was *remove tag T*, and removing `spreadAll` changes the SPREAD set, so the digest moves and the tag scores LIVE while `hitsAlly` stays ignored. Tag-level mutation structurally cannot see a param-level ignore — and param-level is the shape that actually bit (WIRE 71 was three routes writing a literal 5). **Amendment: the operator set includes per-param perturbation** (flip booleans, sentinel numerics), which the tag list supplies for free exactly as it supplies tag removal.
1. **__setDB injection would not have reached the engine.** `medicham2-browser.js` builds its tag-derived sets — `SPREAD`, `HITS_ALLY`, the terrain table, the priority-block map — **at module load** (from line 145). Injecting a mutated DB after load leaves those sets built from the unmutated artifact: the mutation silently no-ops and the tag scores "read-and-ignored" — a false DEAD, the direction the project's own doctrine calls dangerous. **Amendment: __setDB ships with a derived-set rebuild hook**, and the harness's known-bad demonstration must include one set-building tag to prove the hook fires.
1. **One fixed seed is not a battery.** A probabilistic effect can fail to roll under a single seed (false DEAD); and removing a tag shifts PRNG consumption so downstream state diverges for unrelated reasons (false LIVE — benign but confidence-inflating). **Amendment: a small per-mutant seed battery, with both error directions named in the artifact.** The design's INERT arm — ask whether the *reference engine's* two arms differ before scoring — is kept unchanged; it is the equivalent-mutant defense from the mutation literature and is the best idea in the original.

5. Two holes in the registry and generator layers

1. **The registry covered only half the unified rulebook.** "A tag with no handler fails at load" guards the tags output — but `move-effects.json` fields have no registry, and that file is the one the engine actually reads for flinch/status/secondary. **Amendment: exhaustiveness runs over the generator's unified fact model** — every emitted fact has a consumer in at least one output, or a named reason — not over the tags output alone.
1. **The 35 duplicated move tags would have broken registry day one.** Layer 0 clears the 26 ability/item orphans, but the other 35 DEAD move tags (facts duplicated in the second rulebook) still have no handlers — 35 load failures, instant stub pressure, the exact trap the plan named and then left armed. **Amendment: the unified generator emits a carried-by-other-output table and the registry honors it** — the declaration is generated, not hand-maintained, so it cannot rot the way a hand list rots.

Plus one operational amendment: **fail-at-load needs an ops story**. A tags regeneration that derives a new tag bricks the live bot at startup and the site page — loud beats silent, but there must be a pre-deploy smoke in `run-all` and a message that says *why* the bot is absent. A refusal nobody can see is the old failure mode wearing the new fix.

6. The endgame was wrong, and the proof is arithmetic

"Done" item 6 — the exploitability re-run at 58 dimensions, the number owed since the retraction — **should not be launched as written**. The step-rule probe (`data/exploit-step-probe.json` , stamped to release `d3d04b669e18`) proved the fixed rule correct on a planted optimum and simultaneously proved the search cannot move at any affordable budget:

- one accepted step at $d=58$ is worth **0.202 win-rate points**, against a measurement resolution of **4.77 points** at 220 games (independent seeds, as the void run ran);
- at the void run's budget (24×220 games) the fixed rule closes **0.0% ± 0.1** of the distance — and the legacy rule climbed *backwards* (−1.5%);
- the cheapest split that closes even 25% of the distance costs **~960,000 games**;
- the largest family searchable at the old 5,280-game budget is **4 parameters**.

Re-running would burn a week of compute to produce a number statistically indistinguishable from "no search happened," and we would be tempted to quote it. **The amended endgame: reparameterize MAG's policy to a 4–8-number family first, then search that space against a fresh release.** ABRA still has no exploitability number, and saying so remains correct until that run exists.

7. What was deliberately not changed

- The **ratchet asymmetry** (DEAD ratchets, UNREACHED and STAGED never do) — principled and kept.
- The **standing gate** — no check is trusted until shown failing on known-bad input. Tonight it was vindicated again: all 13 wire probes were demonstrated red against a deliberately broken in-memory engine before their green was believed.
- **Layer 0 first** — the matrix is the instrument that verifies everything else; it stays first and is now nearly done (13 open cases, all named).
- **Layer 1 is built but demoted from urgent to insurance**. Two clashes, both now handled at the consumer; the ratchet holds the count at ≤ 2 . The remaining real exposure is the **27 tag-only and 166 fx-only facts that could not be compared at all** — closing that blind mass is what the unified generator is actually for.

8. The amended order of work

1. **Layer 0 (finish)**: the 13 remaining matrix cases; triage of the 26 orphan ability/item tags into covered-by-name / genuinely-missing / redundant; the 5 census gaps.
2. **Layer 1**: one generator, two outputs, byte-identical gate, plus the `carried-by-other-output` table (§5.2) — and the comparison extended into the 193 currently-uncompared facts.
3. **Layer 2 (was 3) — mutation**: per-tag AND per-param operators, `__setDB` + rebuild hook, seed battery, INERT arm; validated on `hitsAllly` and on one set-building tag.
4. **Layer 3 (was 2) — registry**: fail-at-load over the unified fact model, generated declarations, delete-a-handler refuses to start, ops smoke + loud absence message.
5. **Layer 4**: differential and matrix, unchanged, re-run at `--full` after the layers land.

6. **Endgame:** reparameterize MAG (4–8 dims), cut a fresh release, and run the exploitability search that can actually move. That number is still the point; the route to it changed.

Every amendment above is applied to `docs/ENGINE-COVERAGE-PLAN.md` and the Tier-2 spec in `docs/TAG-COVERAGE.md` in the same pass as this document, with the CHANGELOG entry at 3.40.0.